Memory-Efficient Token Permutation

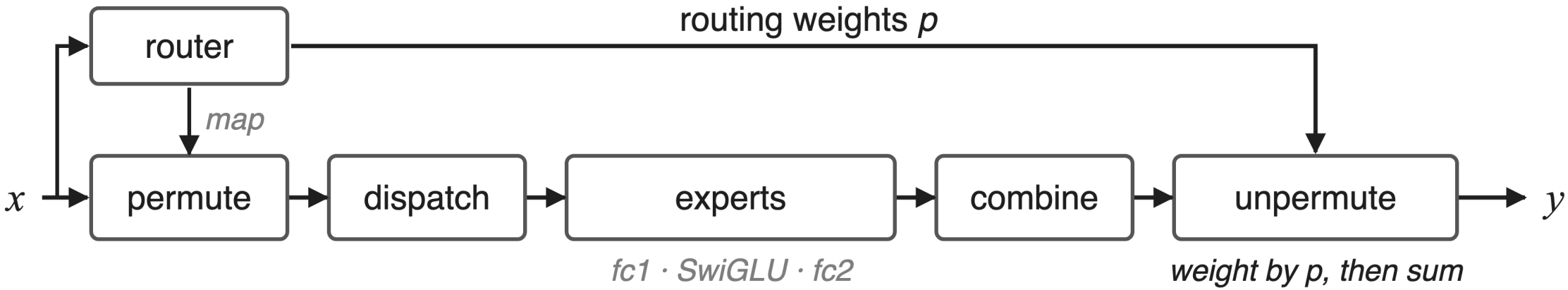
Apply router weights before fc2, not after the expert

NVIDIA · Megatron-Core MoE (v0.13+)

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Memory & Offload Scheduling · github.com/NVIDIA/Megatron-LM

Every MoE output is a router-weighted expert sum



$$y = \Sigma_i \, p_i \cdot E_i(x), \qquad E_i(x) = W_2 \, \varphi(W_1 \, x)$$

DeepSeek-V3: 256 routed experts, top-8 per token. Permute groups tokens by expert; unpermute restores order and combines

Permutation: how tokens and weights reach experts and return

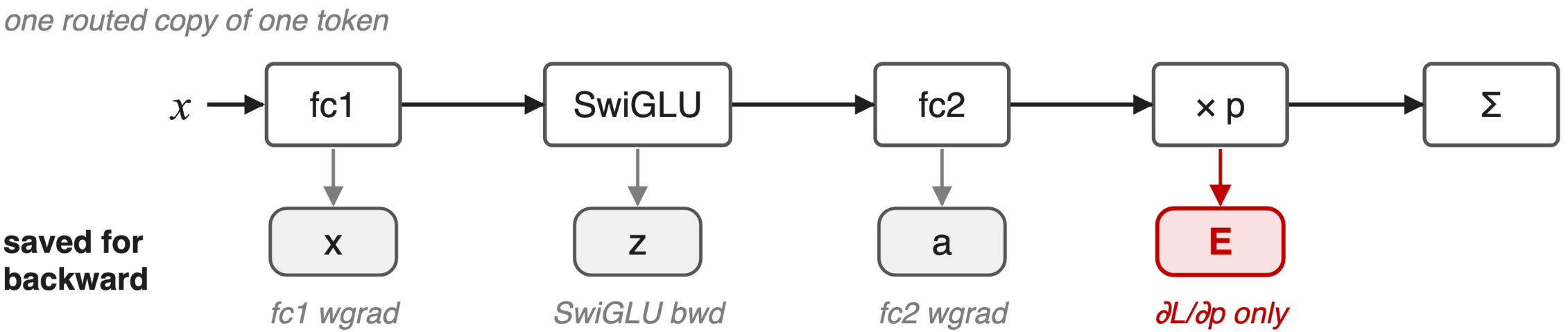
One multiply pins a fifth of all activations

26.3 GB

per GPU of DeepSeek-V3 activations kept only to differentiate the router weights, 20% of the 131 GB (derived)

Megatron-Core MoE technical report, arXiv 2603.07685, §4.1.2 (p.21) and Table 3 (p.20)

Weighting after the expert pins its output



E: one hidden-size row per routed copy

$$\partial L/\partial p = \langle\, E,\, \partial L/\partial y \,\rangle$$

fc2's backward needs its input a, never its output E

Storing or recomputing E both cost something

- **Store E** (standard)
- One hidden-size row per routed copy
- **26.3 GB** per GPU on DeepSeek-V3

- **Recompute E** in backward
- Kimi K2 recomputes its MoE down-projections
- **An extra expert GEMM** per layer

Needed: a router gradient that never touches E

Insight: a scalar weight commutes with fc2

(1) $$y = \Sigma_i \; \boldsymbol{p_i} \cdot W_2 \, \varphi(z_i)$$

(2) $$y = \Sigma_i \, W_2 \, (\boldsymbol{p_i} \cdot \varphi(z_i))$$

valid when W_2 is linear, no bias

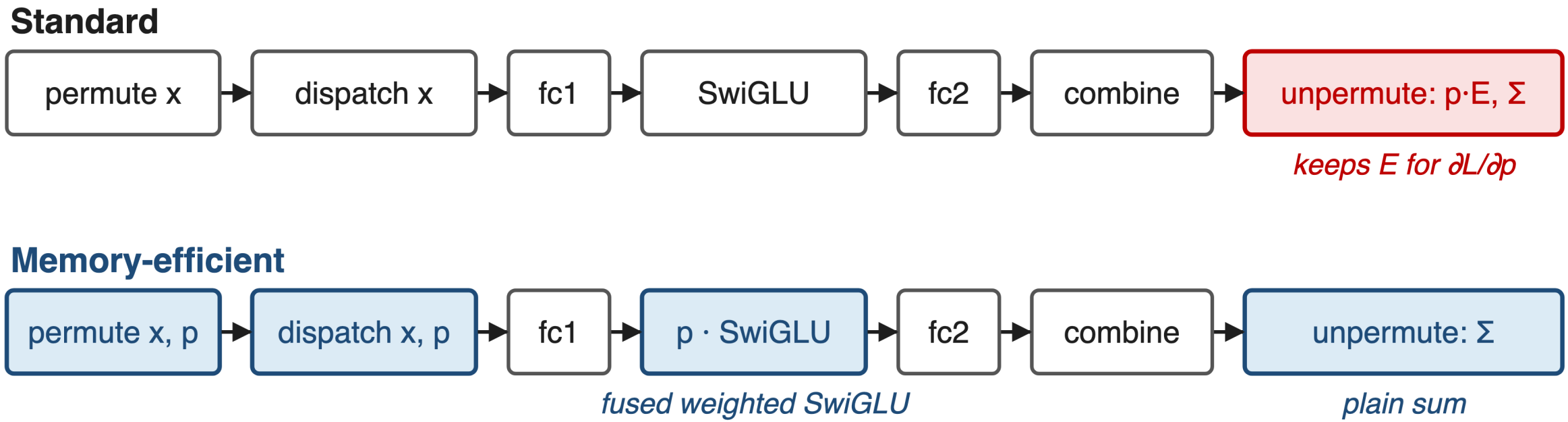
$$\boldsymbol{\partial L / \partial p_i} = \langle \, \varphi(z_i), \, \partial L / \partial a_i \, \rangle$$

ϕ(z) rebuilt from z in the kernel

z is saved for SwiGLU anyway

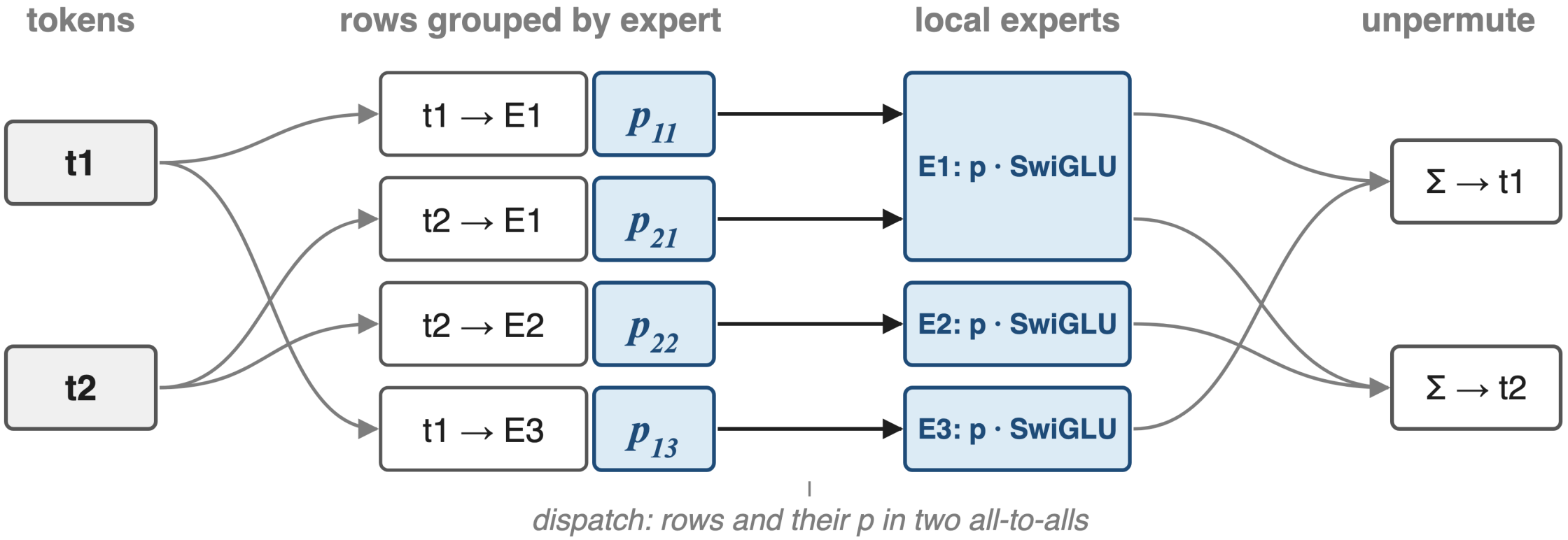
Then the router gradient needs only z, which SwiGLU saves anyway

Move the multiply from unpermute into SwiGLU



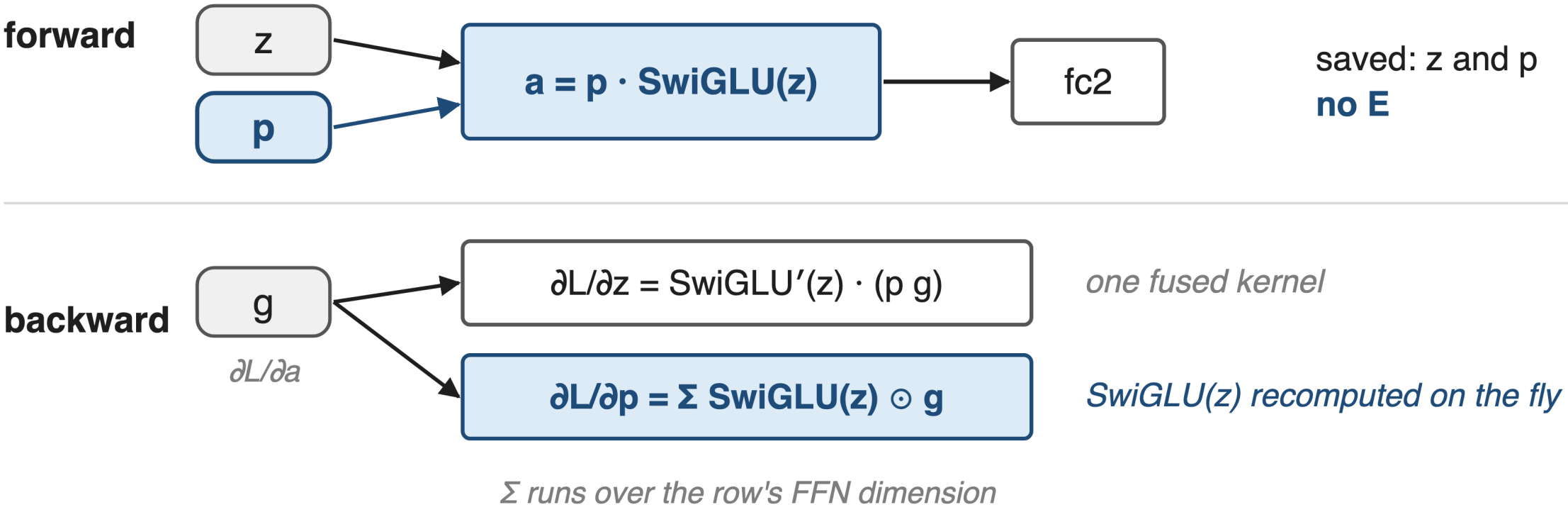
No flag: from MCore v0.13 every expert backend applies probs in its activation; unpermute only sums (§4.3.3)

❶ Router weights travel with their tokens



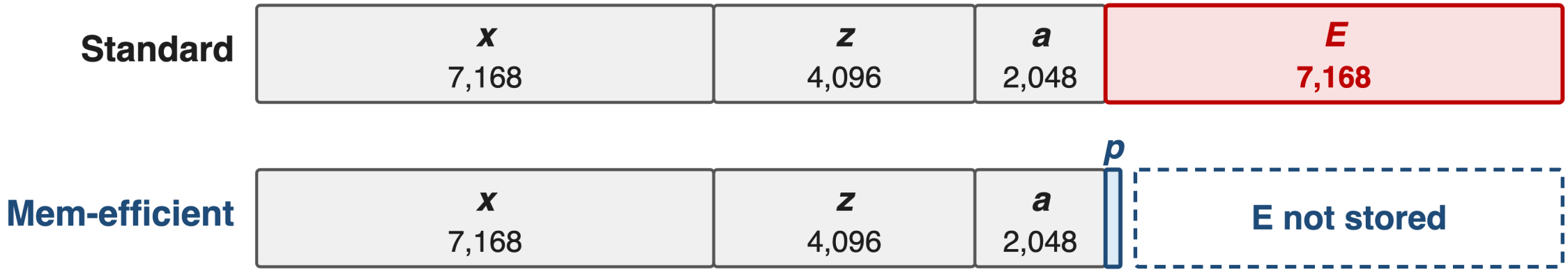
`permute(tokens, routing_map, probs=probs)` → one p per row; `unpermute(...)` without probs. Same for all-to-all, all-gather, DeepEP and HybridEP paths

❷ One fused kernel weights forward and backward



`weighted_bias_swiglu_impl` (megatron/core/fusions/fused_bias_swiglu.py) saves only z and p

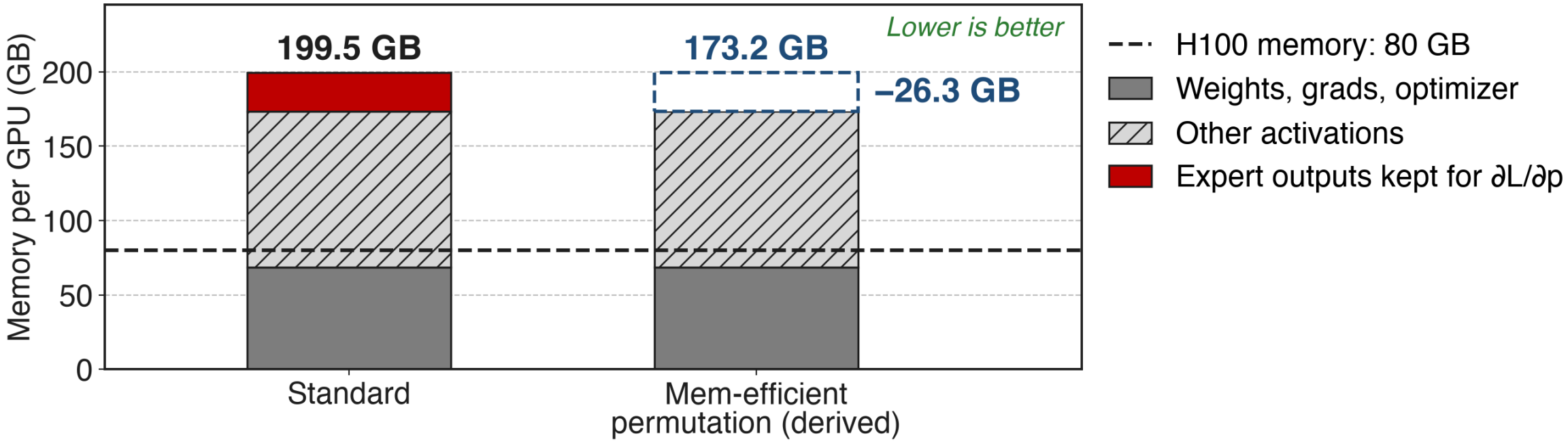
Per routed token, the E row disappears



Analytic, DeepSeek-V3 dimensions (hidden 7,168; expert FFN 2,048, gated fc1 4,096); BF16 rows, p is one scalar

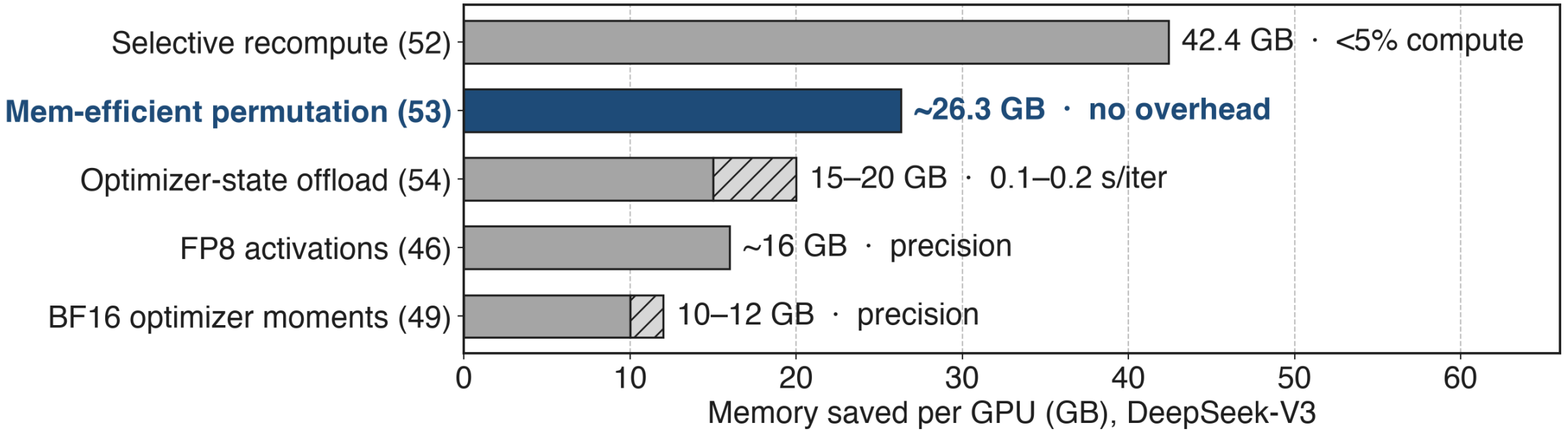
E was 35% of the expert path's saved values per routed copy (analytic)

26.3 GB freed per GPU at zero compute cost



Zero overhead: p is applied once either way, only earlier

The only memory saver with no trade-off



Each saving is reported separately against the 199.5 GB BF16 baseline (Table 3); they do not simply add up

On in both final DeepSeek-V3 configurations, GB200 and H100

Kimi K3 cuts the same dependency another way

- **Megatron-Core**: move p before fc2
- Router gradient from SwiGLU(z)
- **No extra work**

- **Kimi K3**: rewrite the p gradient
- Uses activation output and output gradient
- One light elementwise op

Both stop backward from reading the expert output

The cost: a bias rule and a tiny all-to-all

- Needs a **bias-free fc2**, or a bias scaled by p apart
- Only E goes; x, z, a stay
- p needs **its own all-to-all**, one scalar per row
- Top-1 variant: `moe_apply_probs_on_input`

Pair with moe_act recompute (52), FP8 (46), offload (54)

Summary

- **26.3 GB** per GPU held only for the router-weight gradient
- **Insight**: p is a scalar and fc2 is linear, so weight before fc2
- Probs ride with tokens; one fused weighted-SwiGLU kernel
- **26.3 GB** freed per GPU, **zero** extra compute

https://arxiv.org/abs/2603.07685